

CHARUSAT Online Course Chatbot

Presentation Guide for Demonstration

A RAG-based question-answering assistant for CHARUSAT's online degree programmes (BBA, BCA, MBA, MCA) built with FastAPI, ChromaDB, FastEmbed and a Groq-backed LLM.

Contents

1. Project overview + tech stack
2. Startup commands
3. Main architecture diagram
4. RAG workflow diagrams
5. Most important backend files
6. Vector DB + embedding explanation
7. One complete question flow
8. Unrelated / ignore files
9. Presentation cheat sheet + likely viva questions

1. PROJECT OVERVIEW

What the project is

A RAG (Retrieval-Augmented Generation) chatbot that answers questions about CHARUSAT University's online degree programmes. It scrapes the official website (charusat.online) and reads official programme PDFs, stores the content as vector embeddings in ChromaDB, and answers by retrieving the most relevant chunks and feeding them to an LLM.

What the user sees

- A React web chat UI (in the deployment/ folder) or a terminal chatbot (python main.py).
- The user types a question, e.g. "What is the fee for Online BBA?", and the assistant answers from the knowledge base.

Two entry points

A) Terminal chatbot -> python main.py
B) Web API (FastAPI) -> uvicorn api:app --port 8000 (used by the web UI)

The web UI (deployment/src/App.jsx) calls the FastAPI backend, which is the path your professor is most likely to see. Both paths share the same RAG pipeline inside backend/app/.

2. TECH STACK

Layer	Technology used in this project
Frontend	React 19 + Vite (deployment/ folder is the real chat UI)
Backend API	FastAPI + uvicorn (backend/api.py)
RAG orchestration	LangChain (langchain-core, langchain-chroma) - app/rag_chain.py
Embeddings	BAAI/bge-small-en-v1.5 (384 dims) via FastEmbed ONNX
Vector database	ChromaDB (local folder backend/vector_db/, ~408 vectors)
Chunking	RecursiveCharacterTextSplitter (600 chars, 100 overlap)
LLM (primary)	Groq API - llama-3.3-70b-versatile
LLM (fallback)	Groq failover models, then a deterministic local answer
Knowledge base	Markdown scraped from charusat.online + official PDFs in knowledge_base/
Deployment	Render (backend) + Vercel (frontend)

3. STARTUP COMMANDS

Backend (from the backend/ folder)

```
cd backend
python -m venv .venv      # once
.venv\Scripts\activate    # Windows
pip install -r requirements.txt
uvicorn api:app --reload --port 8000
```

Frontend (from the deployment/ folder)

```
cd deployment
npm install
npm run dev
```

URLs to open

- Backend API docs: <http://localhost:8000/docs>
- Health check: <http://localhost:8000/health>
- Chat UI: <http://localhost:5173>

If the vector database needs rebuilding

```
python rebuild_vectordb.py # website markdown -> ChromaDB
python ingest_pdfs.py      # PDFs -> ChromaDB (adds to existing store)
```

Note

A terminal-only version also works without the web UI:

```
python main.py
```

It loads the same RAG pipeline and answers interactively.

4. MAIN ARCHITECTURE DIAGRAM

How one question travels through the system (actual files)

```
USER (browser)
| POST /chat {question, session_id}
v
React chat UI (deployment/src/App.jsx -> sendMessage())
| fetch(API_URL + '/chat')
v
FastAPI endpoint (backend/api.py -> async def chat())
| ConversationStore (app/conversation_history.py)
v
RAGChain.answer_with_history() (app/rag_chain.py)
| resolve_followup_query() (app/query_resolver.py)
v
RAGRetriever.retrieve() (app/retriever.py)
| semantic search + keyword/phrase/source-authority boosts
v
ChromaDB (backend/vector_db/chroma.sqlite3)
| top-K relevant chunks (K=4)
v
Context builder + conflict detection (app/conflict.py)
v
Prompt (app/prompt.py -> build_prompt_with_history())
v
LLMClient.generate() (app/llm.py)
| Groq llama-3.3-70b-versatile (or local fallback)
v
Answer text (sanitized, no source refs) -> JSON back to React UI
| {session_id, answer}
v
USER sees the answer
```

5. RAG WORKFLOW DIAGRAMS

Offline part: how the knowledge base becomes the vector database

```
Documents
| knowledge_base/ : scraped markdown (home.md, programs/online_bba.md ...)
| + PDFs (PPR_Online BBA.pdf, Fees Refund Policy.pdf ...)
v
Chunking (app/ingestion.py split_documents, app/pdf_ingestion.py)
| RecursiveCharacterTextSplitter - 600 chars, 100 overlap (PDF: 800/1200)
v
Embeddings (app/embeddings.py - FastEmbed, bge-small-en-v1.5, 384 dims)
| each chunk becomes a 384-number vector
v
ChromaDB (backend/vector_db/)
| vectors + chunk text + metadata stored for fast similarity search
v
vector_store (loaded at startup by api.py / chatbot.py)
```

Online part: how a user question gets an answer

User question

| e.g. "What is the fee for Online BBA?"

v

Query embedding (same bge-small model, app/embeddings.py embed_query)

v

ChromaDB search (retriever.py similarity_search_with_relevance_scores)

| programme detected: online_bba -> search scoped to that programme

v

Relevant chunks (candidate pool 20 -> scored -> dedup -> top 4)

v

Prompt + context (app/prompt.py: system rules + retrieved context + question)

v

LLM (app/llm.py: Groq llama-3.3-70b-versatile)

v

Answer (sanitized -> returned to React UI)

6. MOST IMPORTANT BACKEND FILES

File	Purpose	When / why it is used
backend/api.py	FastAPI app + POST /chat endpoint	Runs when the server starts; every question hits chat()
app/chatbot.py	Terminal chatbot (ChatbotApp)	Only for python main.py CLI mode
app/rag_chain.py	RAGChain - composes retrieval, conflict check, prompt, LLM	answer_with_history() is the heart of every answer
app/retriever.py	RAGRetriever - query analysis, Chroma search, scoring, dedup (uses app/deduplicate.py + app/source_authority.py)	Called by rag_chain for every question
app/embeddings.py	FastEmbeddings (bge-small-en-v1.5) + EmbeddingStore (Chroma)	Loads/builds the vector DB; embeds the query
app/llm.py	LLMClient - Groq primary + local deterministic fallback	Generates the final answer text
app/prompt.py	System prompt + build_prompt_with_history	Assembles the prompt sent to the LLM
app/ingestion.py	KnowledgeBaseIngestor - load + chunk markdown	Only when building/rebuilding the vector DB
app/pdf_ingestion.py	PDFIngestor - extract text + tables from PDFs	Only when running ingest_pdfs.py
app/query_resolver.py	Rewrites follow-up questions into standalone queries	Only for follow-ups in the same session
app/conversation_history.py	In-memory session history store	POST /chat session management
app/conflict.py	Detects contradictory values across sources	Adds a Conflict Notice to the context when needed
app/config.py	All settings: chunk size, TOP_K=4, thresholds, models, API keys	Read by every module
knowledge_base/	Markdown + PDFs that are embedded into ChromaDB	Source of all answers
vector_db/	The actual ChromaDB store (~408 vectors)	Loaded at startup

7. VECTOR DATABASE + EMBEDDINGS

Embedding technology (confirmed from app/embeddings.py)

- Primary: BAAI/bge-small-en-v1.5 - 384-dimensional vectors, run through FastEmbed (ONNX runtime) so it uses ~80-120 MB RAM.
- Fallback: all-MiniLM-L6-v2 via HuggingFaceEmbeddings if FastEmbed fails.
- Same model is used to embed documents at build time AND to embed the user query at query time - that is what makes similarity search possible.

Vector database (confirmed from code + vector_db/)

- ChromaDB, stored locally in backend/vector_db/chroma.sqlite3.
- Collection: "langchain" (default Chroma collection name).
- Current size: 408 embeddings/vectors found in the local Chroma file. (An older smoke-test report mentioned 444 chunks.)
- Storage: each chunk is stored as (embedding vector, original text, metadata like source, page, program_name, chunk_id).

How retrieval and ranking work (app/retriever.py)

- The query is classified into a programme (BBA/BCA/MBA/MCA) and an intent (programme / overview / general).
- ChromaDB returns a candidate pool (k=20) using cosine similarity via similarity_search_with_relevance_scores.
- Each candidate gets a final score = base similarity + keyword bonus + value-proximity bonus + verbatim phrase bonus + source-authority bonus (capped at 0.95).
- Near-duplicates are removed (Jaccard similarity > 0.82) and per-page diversity is capped (max 2 per source page).
- The top 4 chunks (TOP_K=4) above the relevance floor (0.52) go to the LLM.

Hallucination control

- System prompt: answer ONLY from the retrieved context, never invent facts (app/prompt.py).

- If nothing relevant is found, a fixed not-found answer is returned instead of a guess.
- Numeric safety rules - the LLM must not do arithmetic the document does not state.
- Conflict detection reports both values when sources disagree.
- Final answer is sanitized so internal file/page/score references never reach the user.

Example: "What is the fee for Online BBA?"

Where the answer comes from: the knowledge base contains programs/online_bba.md which states "Annual Fee: 50,000 INR" - the retriever finds exactly this chunk.

User types: "What is the fee for Online BBA?"

```

|
v deployment/src/App.jsx (sendMessage())
  fetch(POST http://localhost:8000/chat, body {session_id, question})
|
v backend/api.py (async def chat())
  - get_or_create_session() -> new UUID
  - get_recent_history() -> []
  - rag_chain.answer_with_history(question, history)
|
v app/query_resolver.py
  needs_resolution() -> False (self-contained) -> question unchanged
|
v app/retriever.py (RAGRetriever.retrieve())
  - detects programme "online_bba", intent "programme"
  - ChromaDB search scoped to online_bba + general + policy docs (k=20)
  - scores chunks: fee keyword match + "annual fee" phrase + PDF/website authority
  - dedup + diversity cap -> top 4 chunks
|
v app/conflict.py (detect_conflicts_from_documents -> no conflict)
|
v app/rag_chain.py (_build_context: Source/Page/Chunk/Content blocks)
|
v app/prompt.py (build_prompt_with_history: system rules + context + question)
|
v app/llm.py (LLMClient.generate)
  Groq llama-3.3-70b-versatile reads the context and answers:
  "The Annual Fee for the Online BBA is 50,000 INR."
|
v strip_internal_references() -> clean answer, no file names
|
v api.py stores the turn and returns {session_id, answer}
|
v React UI renders the answer bubble
USER sees: The Annual Fee for the Online BBA is 50,000 INR.

```

What if the LLM fails?

If Groq is unavailable (no key, rate limit, timeout), LLMClient falls back to a deterministic local answer that extracts the exact fee value from the retrieved chunks, so the demo still works offline.

9. SEPARATE / UNRELATED PROJECT + IGNORE FILES

SkillForge AI (a second feature in the same repo)

The repository also contains a separate career-mentoring feature called SkillForge AI (resume/portfolio analysis, skill-gap, career plans). It is NOT part of the Online Course Chatbot and is NOT required for your demonstration.

- Files that belong only to SkillForge: backend/app/career_engine.py, portfolio_analyzer.py, resume_analyzer.py, database.py (MongoDB), schemas.py, gemini_client.py.
- backend/api.py ALSO registers SkillForge endpoints (/api/career/*) - ignore those endpoints during the demo.
- Decision: ignore SkillForge during your presentation; focus only on the /chat flow.

Files you can ignore during the presentation

File / folder	Classification
frontend/ (default Vite starter - a counter demo, not the chat UI)	Not used by the chatbot (the real UI is in deployment/)

backend/ingestion.py (old wrapper, superseded by rebuild_vectordb.py)	Old / duplicate
backend/eval_rag.py, run_24_question_test.py, tests/, backend/test_*.py, deployment/smoke_test.py, backend/smoke_test_backend.py	Development / testing only
*.txt and *.md notes: CHANGES.txt, completed-work-summary.txt, terminal_output_summary.txt, today-progress.md, walkthrough.md, implementation_plan.md, DEPLOYMENT_TROUBLESHOOTING_LOG.txt	Development notes - not used by the app
pdf-forge-exports/ (empty)	Not used by the chatbot
requirements-updated.txt, root package-lock.json, root vercel.json	Not used / leftover
SkillForge app files listed above	Separate SkillForge feature
vector_db/ and knowledge_base/ and backend/.env	IMPORTANT - DO NOT delete (required at runtime)
backend/app/ pipeline modules, api.py, deployment/	IMPORTANT - DO NOT delete (core chatbot)

Project in one sentence

A RAG chatbot for CHARUSAT's online programmes that scrapes the university website into a knowledge base, embeds it into ChromaDB, and answers questions with Groq's LLM using only the retrieved evidence.

Complete workflow in one line

Question -> POST /chat (api.py) -> RAGChain.answer_with_history (rag_chain.py) -> retrieve from ChromaDB (retriever.py) -> build prompt (prompt.py) -> Groq LLM (llm.py) -> answer back to the React UI (App.jsx).

Key numbers to quote

- Embedding model: BAAI/bge-small-en-v1.5, 384 dimensions (FastEmbed).
- Vector DB: ChromaDB - ~408 vectors in backend/vector_db/.
- Chunking: 600 chars with 100 overlap (PDFs 800 / tables 1200).
- Retrieval: candidate pool 20, final TOP_K = 4, relevance floor 0.52, dedup threshold 0.82.
- LLM: Groq llama-3.3-70b-versatile (temperature 0.2), with deterministic local fallback.

5 things to explain during the demo

1. What RAG is: retrieve relevant documents, then let the LLM answer only from them.
2. Build pipeline: website + PDFs -> chunk -> embed -> ChromaDB (rebuild_vectordb.py).
3. Query pipeline: embed question -> Chroma search -> top-4 chunks -> prompt -> LLM.
4. Hallucination control: grounded prompt, relevance threshold, not-found fallback, conflict notices.
5. Two interfaces: web UI (React/Vite) and terminal (python main.py).

10 likely professor questions + short answers

Question	Short answer
What is RAG?	Retrieval-Augmented Generation: fetch relevant documents for the question, then let an LLM answer using only that context.
Why do you need embeddings?	To convert text into numbers so the computer can measure semantic similarity and find the most relevant chunks.
Which embedding model?	BAAI/bge-small-en-v1.5, 384-dim vectors, run via FastEmbed (ONNX) for low RAM.
Which vector database and why?	ChromaDB, local and free; fast approximate similarity search without any external DB.
How many chunks are stored?	~408 vectors in the local ChromaDB store.
How does retrieval work?	Query is embedded, ChromaDB returns 20 candidate chunks, they are re-scored with keyword/phrase/source bonuses, de-duplicated, and top 4 selected.
Which LLM generates the answer?	Groq's llama-3.3-70b-versatile; if it fails, a deterministic local answer is generated from the chunks.
How do you stop the LLM from hallucinating?	System prompt forces answers from context only, relevance threshold, not-found fallback, and numeric safety rules.
What is TOP_K?	4 - the number of most relevant chunks sent to the LLM as context.
How does it handle follow-up questions?	query_resolver.py rewrites follow-ups into standalone queries using conversation history; history is also added to the prompt.